

Arduino Cookbook : De la Théorie à la Pratique

Chapitre 1 : Mise en Route (Getting Started)

Présenté par : Dr B. DIOP

29 mars 2026

Référence : Michael Margolis, *Arduino Cookbook*, 2nd Edition, O'Reilly

1.0 Introduction : L'écosystème Arduino

Qu'est-ce que l'Arduino ? (Margolis, p.1)

L'Arduino est une plateforme de prototypage électronique open-source basée sur du matériel et des logiciels flexibles et faciles à utiliser. Elle est destinée aux artistes, designers, amateurs et à toute personne intéressée par la création d'objets ou d'environnements interactifs.

- **Physical Computing** : L'Arduino permet de concevoir des systèmes capables de percevoir leur environnement (capteurs) et d'y réagir (actionneurs : moteurs, lumières).
- **Le microcontrôleur** : Le cerveau de la carte (souvent un ATmega328). Il exécute les instructions programmées.
- **L'IDE** : L'environnement de développement intégré, utilisé pour écrire et transférer le code ("sketch").

Illustration : Insérer la Figure 1-1 (Carte Arduino Uno et ses composants).

1.1 Recette : Installation de l'Environnement Intégré (IDE)

Problème : Vous devez installer le logiciel pour programmer votre carte.

Solution : Télécharger l'IDE depuis `arduino.cc`.

Détails par système d'exploitation :

- **Windows :** L'installateur installe automatiquement les pilotes USB. Si la carte n'est pas reconnue, il faut passer par le Gestionnaire de périphériques pour cibler manuellement le dossier `drivers` de l'IDE.
- **macOS :** Une simple application à glisser dans le dossier Applications. Aucun pilote FTDI supplémentaire n'est requis pour la Uno.
- **Linux :** Installation via le gestionnaire de paquets (ex : `sudo apt-get install arduino`) ou via l'archive officielle. Nécessite d'ajouter l'utilisateur au groupe `dialout` pour les permissions du port série.

1.2 Recette : Configuration de la carte et du port série

Problème : L'IDE ne parvient pas à communiquer avec l'Arduino.

Solution : Sélectionner la bonne carte et le bon port de communication.

1. **Choix de la carte (Board)** : Aller dans *Tools > Board*. Sélectionner "Arduino Uno" (ou Mega, Leonardo, etc., selon le matériel). Si le mauvais modèle est sélectionné, la compilation échouera.
2. **Choix du port série (Serial Port)** : Aller dans *Tools > Serial Port*.
 - Sous Windows : Ce sera un port COMx (généralement COM3 ou supérieur).
 - Sous macOS/Linux : Ce sera `/dev/tty.usbmodem...` ou `/dev/ttyACM0`.

Attention

Débrancher puis rebrancher la carte USB permet d'identifier facilement quel port apparaît/disparaît dans la liste.

1.3 Recette : Structure fondamentale d'un Sketch

Problème : Comprendre l'anatomie d'un programme (Sketch) Arduino.

Solution : Un sketch Arduino requiert obligatoirement deux fonctions C/C++.

```
// 1. Zone de declaration globale (variables, bibliotheques)
int ledPin = 13;

// 2. Fonction d'initialisation (executee une seule fois)
void setup() {
    // Configuration matérielle, ex: definir une broche en SORTIE
    pinMode(ledPin, OUTPUT);
}

// 3. Fonction principale (executee en boucle a l'infini)
void loop() {
    // Logique du programme
}
```

Note de l'auteur (Margolis) : L'Arduino n'a pas de système d'exploitation pour arrêter le programme ; c'est pourquoi la fonction loop() est infinie.

1.4 Recette : Écrire et comprendre le sketch "Blink"

Problème : Vérifier que la chaîne matérielle et logicielle fonctionne.

Solution : Charger le programme "Blink" (le "Hello World" matériel).

```
const int ledPin = 13; // La broche 13 a une LED integree sur la Uno

void setup() {
  pinMode(ledPin, OUTPUT); // Configure la broche 13 comme une sortie
}

void loop() {
  digitalWrite(ledPin, HIGH); // Allume la LED (5 Volts)
  delay(1000);                // Attend 1000 millisecondes (1 seconde)
  digitalWrite(ledPin, LOW);  // Eteint la LED (0 Volt)
  delay(1000);                // Attend 1 seconde
}
```

Illustration : Insérer le schéma de la LED connectée à la broche 13 (Figure 1-5).

Dans le sketch "Blink", nous introduisons trois fonctions critiques de l'API Arduino :

- `pinMode(pin, mode)` : Indique au microcontrôleur si la broche (*pin*) doit se comporter comme une entrée (INPUT) pour lire des capteurs, ou comme une sortie (OUTPUT) pour fournir du courant.
- `digitalWrite(pin, valeur)` : Modifie l'état électrique de la broche. HIGH connecte la broche à l'alimentation (5V), fournissant du courant. LOW connecte la broche à la masse (0V), coupant le courant.
- `delay(ms)` : Suspend l'exécution du programme pendant le nombre de millisecondes spécifié. **Attention** : Pendant un `delay()`, l'Arduino ne peut rien faire d'autre.

1.5 Recette : Le processus de Compilation et de Téléversement

Problème : Comment transformer du code C++ lisible par l'homme en impulsions électriques dans le microcontrôleur ?

Le processus (Background) :

1. **Vérification (Bouton Tick) :** Le compilateur (avr-gcc) vérifie la syntaxe. S'il y a une erreur (ex : un point-virgule manquant), le code n'est pas compilé.
2. **Création de l'hexadécimal :** Le code est traduit en langage machine (un fichier `.hex`).
3. **Téléversement (Bouton Flèche) :** Le programme *avrdude* communique avec le "Bootloader" (petit programme de démarrage) de l'Arduino pour écrire le fichier `.hex` dans la mémoire Flash.

Indicateur visuel : Les LEDs RX (Réception) et TX (Transmission) de la carte Arduino clignotent frénétiquement pendant cette étape.

1.6 Recette : Utilisation du Moniteur Série (Serial Monitor)

Problème : L'Arduino n'a ni écran ni clavier. Comment afficher des variables ou déboguer le programme ?

Solution : Utiliser la communication série via le câble USB.

Configuration requise

Il faut initialiser la communication dans le `setup()` avec `Serial.begin(baud_rate);`. Le "Baud rate" est la vitesse de transmission en bits par seconde. L'IDE et l'Arduino doivent avoir **exactement la même vitesse** (ex : 9600).

Exemple détaillé de Communication Série

```
int compteur = 0; // Variable pour compter les cycles

void setup() {
  Serial.begin(9600); // Initialise le port serie a 9600 bauds
  Serial.println("Demarrage du programme...");
}

void loop() {
  compteur = compteur + 1; // Incrementation

  Serial.print("Le compteur est a : "); // Affiche sans retour a la ligne
  Serial.println(compteur);             // Affiche la valeur et saute une ligne

  delay(2000); // Attend 2 secondes pour ne pas inonder le moniteur
}
```

Action : Cliquer sur l'icône de la loupe en haut à droite de l'IDE pour ouvrir le moniteur.

Erreur : avrdude: stk500_getsync(): not in sync: resp=0x00

- **Cause principale :** L'IDE ne trouve pas l'Arduino au moment du téléversement.
- **Vérifications :**
 1. Le câble USB est-il défectueux ou uniquement conçu pour la charge ?
 2. Avez-vous sélectionné le bon port COM / dev dans les outils ?
 3. Avez-vous des composants branchés sur les broches 0 (RX) et 1 (TX) ? (Ces broches sont partagées avec l'USB, les débrancher pendant l'upload).

Erreur : expected ';' before '}' token

- Le C++ est strict. La majorité des erreurs de débutant proviennent de :
- Oubli de point-virgule (;) à la fin d'une instruction.
- Oubli de fermer une accolade (}).
- Sensibilité à la casse : `pinmode` au lieu de `pinMode`. `digitalwrite` au lieu de `digitalWrite`.

Astuce de l'IDE

L'IDE surligne en rouge la ligne *après* l'erreur. Si la ligne surlignée semble correcte, cherchez l'erreur (souvent un point-virgule manquant) sur la ligne juste au-dessus.

Résumé du Chapitre 1

- **Écosystème** : L'Arduino est la combinaison d'une carte (microcontrôleur) et d'un IDE (logiciel).
- **Structure** : Tout programme nécessite un `setup()` (configuration initiale) et un `loop()` (boucle de contrôle).
- **E/S numériques** : `pinMode()`, `digitalWrite()`, `delay()` sont la base de l'interaction matérielle.
- **Débogage** : Le Moniteur Série (`Serial.begin`, `Serial.print`) est l'outil indispensable pour voir ce que "pense" le microcontrôleur.

Dans le prochain chapitre, nous approfondirons les fondations du langage C++ appliquées à l'Arduino (Variables, conditions, boucles complexes).